

State engineering (NgRx SignalStore)

Trust one source of truth · withEntities · rxMethod · TMS enrollments

TMS command center

```
export const EnrollmentStore = signalStore(  
  { providedIn: 'root' },  
  withState({ isLoading: false, error:  
    null }),  
  withEntities<Enrollment>(), ...  
);
```

One screen, one truth

Session goals

By the end of this session, you will:

- Explain state drift when two components each keep their own copy of the same TMS data (spine)
- Separate local UI state from global authoritative collections when SignalStore wins (spine)
- Name the three pillars: state, computed, methods API ↔ store ↔ MatTable / KPIs (spine)
- Contrast flat Enrollment[] with withEntities dictionary shape and O(1) row updates (spine)
- Assemble signalStore + withState + withEntities + withComputed + withMethods (first encounter)
- Wire rxMethod + concatMap for safe queues; sketch optimistic approve + rollback on failure (spine)
- Avoid Session 1 traps: skipping withEntities, mutating arrays, duplicate startup loads (mistakes table)

The big picture: state drift

Imagine Abeba (TMS Admin) on the enrollment dashboard.

The sidebar says "15 Pending Enrollments". The main table says "13 Pending Enrollments".

Both components called `GET /api/enrollments` but one call failed, or they ran at slightly different times. Because state lived in two separate local `signal()` variables, the UI drifted out of sync.

If a user sees two different numbers on the same screen, they lose trust in the software.

Global state vs local state

State type	Where it lives	Example
Local state	Inside the component (signal())	Is this dropdown open?
Global state	Singleton service / store	Student list; logged-in user shape; enrollment rows

When to use NgRx SignalStore

When several components need to read and write the same mutable data on the same screen not for every toggle (spine).

- M8 local signals are fine for component UX not for authoritative copies of DB-backed grids (spine speaker note)

SignalStore architecture three pillars

- 1) State raw enrollment rows, loading, error flags (spine)
- 2) Computed derived signals that recompute when state changes (e.g. pendingCount)
- 3) Methods actions that mutate via API streams (rxMethod + patchState / entity helpers)

Flow (spine mermaid in prose)

TMS .NET API ↔ rxMethod (concatMap queue) ↔ patchState / updateEntity ↔ withEntities dictionary ↔ MatTable + dashboard KPIs; withState drives spinners and errors on the same components.

withEntities vs plain arrays

- 5,000 enrollments + instructor clicks Approve on one row → `array.findIndex()` scans every time (spine)
- `withEntities<Enrollment>()` keeps `ids[]` + `entities{}` update by id is $O(1)$ lookup (spine)

EntityCollection shape (concept)

```
// Shape stored by withEntities (spine JSON)
{
  "ids": [101, 102],

  "entities": {
    "101": { "id": 101, "status": "Pending" },
    "102": { "id": 102, "status": "Approved" }
  }
}
```

Interview line

Map vs array sweep enterprise grids freeze when every click rescans thousands of rows; `withEntities` is the native fix (spine).

Building the store first encounter

enrollment.store.ts (spine skeleton)

```
export const EnrollmentStore = signalStore(  
  { providedIn: "root" },  
  withState({ isLoading: false, error: null }),  
  withEntities<Enrollment>(),  
  withComputed((store) => ({  
    pendingCount: computed(() =>  
      store.entities().filter((e) => e.status === "Pending").length  
    }),  
  })),  
  withMethods((store, api = inject(EnrollmentService)) => ({  
    load: rxMethod<void>(/* pipe ... */),  
  })),  
);
```

- `signalStore` builds the container
- `withState` loading / error flags (and other custom fields)
- `withComputed` read-only derived signals from entities + state

rxMethod and safe queuing

Pattern (spine trimmed)

```
loadEnrollments: rxMethod<void>(
  pipe(
    tap(() => patchState(store, { isLoading: true })),
    concatMap(() => api.getAll().pipe(
      tap((rows) => patchState(store, setAllEntities(rows), { isLoading: false }))
    ))
  )
)
```

- `patchState` immutable, supported updates to the store (only safe write path)
- `rxMethod` bridges signals with RxJS streams (HTTP, queues)
- `concatMap` queues overlapping calls so they run one at a time (integrity)

Optimistic updates rollback

Optimistic path (spine error handling trimmed)

```
approveEnrollment: rxMethod<string>(  
  pipe(  
    tap((id) => {  
      patchState(store, updateEntity({ id, changes: { status: 'Approved' } }));  
    }),  
  
    concatMap((id) => api.approve(id).pipe(  
      catchError(() => {  
        patchState(store, updateEntity({ id, changes: { status: 'Pending' } }));  
        return EMPTY;  
      })  
    ))  
  )  
)
```

- Optimistic UI flips to Approved immediately; server still in flight
- Rollback on failure, patchState restores Pending and surface an error banner
- Same pattern as high-frequency social UIs hide latency where safe (spine)

Common mistakes Session 1

Mistake	Symptom	Fix
Skipping withEntities	Poor performance on large grids when updating rows	Always use @ngrx/signals/entities for collections
Direct array mutation	App data disconnected from Angular reactivity; UI does not repaint	Use patchState to update the store safely
Duplicate API calls	Component A and B both call .load() explicitly on startup	Fetch once at router level, or check if loaded is false before fetching

Session 1 summary

- Drift = multiple local copies of the same enrollment truth
- SignalStore + withEntities = one dictionary-backed source for grids and KPIs
- rxMethod + concatMap + patchState = safe HTTP queues and immutable updates
- Optimistic UI + rollback = fast feel when the server usually agrees

Next: Session 2 Performance and enterprise grids

@defer, bundle discipline, MatTable at TMS scale less code shipped before first paint.